SPM Modular Project

Module 3:Partitioned Hash Join with Duplicates with OpenMP

Student: Karol Jinet Cardona Bolanos

Mat. 702023

Academic Year 2025/2026

1 Design Choices

Common baseline: The sequential reference, the partition-mapping function (`multiply_shift`) and the build-phase data structure (`FlatCountMap`) are inherited unchanged from Module 2. This guarantees that any performance difference observed in this module is attributable to the parallel runtime, not to algorithmic improvements. The same parameters are reused: NR = 50M, NS = 80M, $P = 256$, max-key = 10^6.

OpenMP variants: Two OpenMP versions of the algorithm are implemented, deliberately different in how the parallelism is expressed:

- `hashjoin_omp_loop`: histogram and scatter use `#pragma omp parallel` with manual chunking (one contiguous slice per thread, matching the C++ thread layout required for a conflict-free scatter); the local-join phase uses `#pragma omp parallel for schedule(runtime)`, so the policy (`static`, `dynamic`, `guided`) is read at runtime from the `OMP_SCHEDULE` environment variable. A single binary therefore covers all the scheduling policies tested in Section 4.
- `hashjoin_omp_task`: the local-join phase spawns one `#pragma omp task` per partition ($P = 256$ tasks for $T = 8 \dots 16$ workers), under a single `omp single` block. The OpenMP runtime distributes the tasks dynamically across the threads. Histogram and scatter use T coarse-grained tasks (one per chunk) for stylistic consistency, since their per-record work is uniform.

Both versions share the same per-thread accumulation strategy: a private `PaddedJoinResult` slot per thread, with `alignas(64)` forcing each slot onto its own cache line to eliminate false sharing. The slots are summed sequentially after the implicit barrier at the end of the parallel region. The C++ thread version from Module 2 (`hashjoin_par`, stride-T round-robin distribution of partitions) is reused as a third reference.

Skewed input generator. A controllable skewed workload is generated by rejection sampling: a pool of 1024 distinct keys per hot partition is precomputed; for each record, with probability F a key is sampled from the hot pool, otherwise a uniform key is drawn. Both R and S share the same hot pool seed so that matches still occur inside hot partitions. The default configuration uses $H = 4$ hot partitions out of $P = 256$ and $F = 0.9$, producing partitions that carry $\frac{0.9/4}{0.1/252} \approx 567\times$ more work than the cold ones

Experimental setup: Benchmarks were executed on `spmcluster` (`--partition=normal`, `--exclusive`), with NUMA pinning via `numactl --cpunodebind=0 --membind=0` and OpenMP threads bound to physical cores via `OMP_PROC_BIND=close OMP_PLACES=cores`. Each binary is rebuilt on the assigned compute node before timing to ensure `-march=native` matches the local instruction set. Every reported value is the median of 3 runs, except for the variance study of Section 6, which uses 30 runs at the (H=4, F=0.9, T=16) configuration.

2 Scalability on Uniform Input

Table 1 reports strong and weak scaling on the uniform workload. Strong scaling uses NR = 50M, NS = 80M, while weak scaling uses $\text{NR}_T = 10\text{M} \cdot T$ and $\text{NS}_T = 16\text{M} \cdot T$, matching the Module 2 setup. The The `loop` version uses `OMP_SCHEDULE=dynamic,1`; the choice of schedule has no measurable effect on uniform input, as established in Section 4.

Table 1: Uniform workload: median total_s and speedup vs. sequential (left); weak-scaling efficiency T_1/T_T (right).

	Strong scaling (speedup)				Weak scaling (efficiency)		
T	seq	par	loop	task	par	loop	task
1	3.496	3.453 (1.01)	3.467 (1.01)	3.401 (1.03)	1.00	1.00	1.00
2	–	2.055 (1.70)	2.004 (1.75)	1.996 (1.75)	0.84	0.85	0.86
4	–	1.238 (2.82)	1.223 (2.86)	1.226 (2.85)	0.66	0.67	0.66
8	–	0.827 (4.23)	0.825 (4.24)	0.818 (4.27)	0.49	0.48	0.49
16	–	**0.706 (4.95)**	**0.707 (4.95)**	**0.710 (4.92)**	0.27	0.26	0.26
32	–	0.723 (4.84)	0.702 (4.98)	0.711 (4.92)	0.14	0.13	0.13

All three implementations are equivalent on uniform input. The spread between any pair never exceeds ∼1% at any thread count, with peak speedup ∼4.95× at $T = 16$. After this point, adding threads offers no further gain in either strong or weak scaling: the data-movement phases saturate the DRAM bandwidth of the local NUMA node, and SMT siblings ($T = 32$) only add scheduling pressure. The trajectory matches the implementation of Module 2 (4.91× speedup, 0.27 weak efficiency at $T = 16$ on the same hardware).

3 Scalability on Skewed Input

Table 2 repeats the experiment with the skewed workload ($F = 0.9$, $H = 4$). The contrast with Table 1 is striking: the choice of OpenMP construct, irrelevant on uniform input, now produces a 1.5–2.5× spread in execution time at $T = 16$.

Table 2: Skewed workload, strong scaling: median total_s and speedup vs. sequential (3 runs per cell). `loop-s` = `omp_loop` with `static`; `loop-d1` = with `dynamic,1`.

T	seq	par	loop-s	loop-d1	task
1	2.751	2.716 (1.01)	2.706 (1.02)	2.703 (1.02)	2.685 (1.02)
2	—	1.599 (1.72)	2.359 (1.17)	1.587 (1.73)	1.577 (1.74)
4	—	0.991 (2.78)	2.159 (1.27)	0.982 (2.80)	0.980 (2.81)
8	—	0.883 (3.11)	2.054 (1.34)	0.870 (3.16)	0.862 (3.19)
16	—	**0.889 (3.10)**	2.051 (1.34)	1.307 (2.10)	**0.876 (3.14)**
32	—	0.895 (3.07)	2.049 (1.34)	1.306 (2.11)	0.881 (3.12)

Loop-static collapses by design: The total time of `loop-s` freezes at ∼2.05 s from $T = 4$ onwards, yielding a maximum speedup of 1.34×. The cause is intrinsic to the policy: `schedule(static)` divides the $P = 256$ partitions into T contiguous blocks, so the four hot partitions $\{0, 1, 2, 3\}$ land in the first block owned by thread 0, which alone processes ∼90% of the records while the other 15 threads finish their cold partitions in milliseconds and idle. The fix is to switch `OMP_SCHEDULE` to `dynamic,1` (analysed in Section 4); other policies do not help, as we will see.

Loop-dyn1 and task scale, but unevenly. Both `par` (stride-T round-robin) and `task` (dynamic work-stealing on $P = 256$ tasks) reach ∼3.1× already at $T = 8$ and stay at that plateau through $T = 32$. The plateau itself is dictated by the four hot partitions: with $H = 4$, no scheduling strategy can distribute hot work to more than four threads in parallel. `loop-d1` matches them at $T = 8$ (3.16×), but its median collapses to 1.307 s (2.10×) from $T = 16$ onwards. This is not a flaw of `dynamic,1` but a manifestation of the bimodal distribution analysed in Section 6: with only 3 repetitions, `loop-d1` consistently lands in the slow mode while `task` lands in the fast mode. The Module 2 implementation avoids this issue entirely: with stride-T round-robin, partition i is always processed by thread $i \bmod T$, so the four hot partitions $\{0, 1, 2, 3\}$ are guaranteed to land on four distinct threads regardless of runtime scheduling decisions.

Table 3: Skewed workload, weak scaling: per-thread base 10M/16M. Efficiency is T_1/T_T for each implementation.

	Time (s)				Efficiency			
T	par	loop-s	loop-d1	task	par	loop-s	loop-d1	task
1	0.552	0.547	0.547	0.543	1.00	1.00	1.00	1.00
2	0.662	0.974	0.652	0.647	0.83	0.56	0.84	0.84
4	0.826	1.846	0.821	0.818	0.67	0.30	0.67	0.66
8	1.495	3.600	1.478	1.467	0.37	0.15	0.37	0.37
16	**3.039**	7.320	4.562	4.589	0.18	0.07	0.12	0.12
32	**6.271**	15.629	9.786	9.779	0.09	0.03	0.06	0.06

Weak scaling amplifies all three effects. The skewed weak-scaling efficiency drops far below the uniform case for every implementation: at $T = 16$ the bottleneck thread processes ∼94 M records on its own, so the per-thread work no longer remains constant as T grows. `loop-s` efficiency drops to 0.07 at $T = 16$ and 0.03 at $T = 32$. `loop-d1` and `task` maintain efficiencies comparable to `par` up to $T = 8$ (~ 0.37 for all three), but at $T \geq 16$ both collapse to 0.12 and 0.06 respectively at $T = 16$ and $T = 32$, versus 0.18 and 0.09 for `par`. The cause is the same bimodality described above: at these thread counts both OpenMP runtimes fall into slow mode in ~ 2 runs out of 3, and the median weak-scaling time inflates by a factor of $\sim 1.5\times$ relative to `par`.

3.1 Phase Breakdown

The total times of Table 2 hide an important detail: not all phases of the algorithm contribute equally to the slowdown. Table 4 decomposes each measurement into the *partitioning* phase (histogram + scatter, applied to both relations) and the *join* phase (build + probe per partition).

Table 4: Phase breakdown on skewed input ($F = 0.9$, $H = 4$), median over 3 runs.

	`par`		`loop-s`		`loop-d1`		`task`	
T	part.	join	part.	join	part.	join	part.	join
1	0.970	1.742	0.971	1.731	0.971	1.729	0.969	1.714
2	0.675	0.921	0.679	1.677	0.678	0.905	0.676	0.897
4	0.508	0.481	0.504	1.650	0.502	0.478	0.504	0.473
8	0.414	0.466	0.415	1.635	0.415	0.451	0.412	0.446
16	0.422	0.464	0.416	1.632	0.416	0.887	0.419	0.453
32	0.431	0.460	0.414	1.631	0.415	0.888	0.424	0.453

The decomposition makes two facts immediate. First, *partitioning is unaffected by skew*: every record contributes the same amount of work to histogram and scatter regardless of which partition it ends up in, and all four implementations scale identically (from ∼0.97 s at $T = 1$ to ∼0.42 s at $T = 16$, after which DRAM bandwidth saturates). The bottleneck of the skewed workload is therefore localised *entirely* in the join phase. Second, the asymptotic join time of ∼0.45 s observed for `par` and `task` is the natural lower bound: it is the time to process the heaviest hot partition on a single thread, and no scheduling strategy can go below it. The 1.63 s plateau of `loop-s` is approximately $4\times$ this value, consistent with the fact that all four hot partitions are serialised on thread 0. The 0.88 s plateau of `loop-d1` at $T \geq 16$ is exactly $\sim 2\times$ the optimum: in slow mode, two of the four hot partitions end up on the same thread, doubling that thread's work.

4 Scheduling Policy Comparison

Section 3 showed that `dynamic,1` is significantly faster than `static` on skewed input. However, `dynamic,1` has the highest synchronisation overhead among OpenMP policies, since it dispatches one iteration at a time. A natural question is whether the cheaper alternatives `dynamic,4` and `guided` achieve the same balance with less overhead. The `schedule(runtime)` clause lets a single binary cover all policies.

Table 5: Effect of `OMP_SCHEDULE` on `omp_loop` at $T = 16$. `omp_task` included as reference; medians of 3 runs.

Policy	Uniform (s)	Skewed (s)	Speedup vs static
static	0.711	2.052	1.00×
dynamic,1	0.705	**1.302**	**1.58**×
dynamic,4	0.710	2.046	1.00×
guided	0.707	2.051	1.00×
omp_task	0.708	1.303	1.57×

On uniform input the policy is irrelevant. All four policies fall within 1% of each other. The workload is already balanced and memory-bound, so the runtime gains nothing from rebalancing.

On skewed input only `dynamic,1` breaks the cluster. `dynamic,4` and `guided` freeze at ∼2.05 s like `static`: chunks of 4 iterations or geometrically decreasing chunks proportional to P/T are still large enough to absorb $\{0, 1, 2, 3\}$ in a single dispatch. Only `dynamic,1` (chunk size 1) decomposes the hot cluster across four distinct threads.

An apparent equivalence with `omp_task`. At $T = 16$ the medians of `dynamic,1` and `omp_task` differ by only 1 ms (1.302 s vs. 1.303 s), suggesting that the two are functionally equivalent on this workload: the task scheduler enforces a work-stealing behaviour very similar to `dynamic,1`'s per-iteration dispatch. The choice between loop and task construct is therefore not what matters, what matters is the scheduling policy underneath. The agreement is only on the median, however: Section 6 show that the underlying distributions differ measurably.

5 Skew Sensitivity

To check the conclusions across different degrees of imbalance, we sweep $H \in \{4, 8, 17\}$ and $F \in \{0.5, 0.7, 0.9\}$ at fixed $T = 16$, comparing `loop_s` and `dynamic,1` against `par` and `task`.

Table 6: Skew sensitivity at $T = 16$. `loop-s` = `omp_loop` with `static`; `loop-d1` = `omp_loop` with `dynamic,1`.

H	F	par	loop-s	loop-d1	task
4	0.5	0.789	1.373	0.973	0.757
	0.7	0.901	1.899	1.231	0.880
	0.9	0.890	2.051	1.305	1.306
8	0.5	0.690	1.304	0.738	0.736
	0.7	0.749	1.843	0.887	0.889
	0.9	0.721	2.010	0.921	0.923
17	0.5	0.742	1.255	0.720	0.721
	0.7	0.836	1.794	0.813	0.822
	0.9	0.833	1.929	0.829	0.839

Loop-static is the systematic loser. For every (H, F) combination, `loop-s` is the slowest implementation, by a factor of 1.5× to 2.7×. Switching to `dynamic,1` recovers most of the gap: in 6 out of 9 cells `loop-d1` matches `omp_task` within 1%. The three exceptions all sit in the $H = 4$ row, where `loop-d1` runs 28–40% slower than `omp_task`. As anticipated in Section 4, this is the same bimodality pattern that will be characterised in Section 6: with $H = 4$ hot partitions and $T = 16$ workers, `dynamic,1` consistently lands in the slow mode while `omp_task` mostly lands in the fast mode.

When $H \geq T$ the imbalance dilutes for the work-balancing implementations. At $H = 17 > T = 16$, the work-stealing implementations (`par`, `loop-d1`, `task`) re-converge to ∼0.82 s, since now there are more hot partitions than threads and the heavy work spreads naturally across all workers regardless of the scheduling decision. `loop-s`, however, remains penalised (1.25–1.93 s): under `static` the 17 hot partitions still fall mostly inside the first two contiguous blocks, so two threads each receive 2 hot partitions while the remaining 14 receive

at most one. The two doubly-loaded threads remain the bottleneck, although the relative penalty is smaller than at $H = 4$ because each hot partition individually weighs less when the skewed mass is spread over 17 destinations instead of 4.

`par` is competitive at $H \leq T$. For $H \in \{4, 8\}$, `par` matches or beats `omp_task`: when the H consecutive hot indices fit within the first T thread slots, the deterministic mapping of `par` (Section 3) places each hot partition on a distinct thread by construction, while `omp_task` needs a fortunate runtime race to do the same.

6 Variance Study: Bimodality under Skew

To characterise the run-to-run dispersion observed in Sections 3–5, we ran 30 repetitions of the (H=4, F=0.9, T=16) configuration for the three parallel implementations.

impl	min	max	mean	fast/slow
`par`	0.888	0.894	0.890	30 / 0
`loop-d1`	0.876	1.327	1.293	1 / 29
`task`	0.874	1.306	0.976	23 / 7

Table 7: Total_s over 30 runs

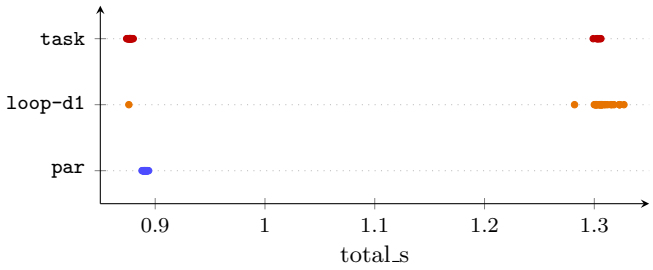


Figure 1: Sorted total_s over 30 runs.

Three distinct distribution shapes. `par` produces a tight unimodal distribution. Both OpenMP implementations are clearly bimodal, with the same two clusters at ∼0.877 s (fast mode) and ∼1.303 s (slow mode), but in inverted proportions: `omp_task` hits fast mode 23/30, `loop-d1` hits slow mode 29/30. The two clusters are perfectly separated: no run produces an intermediate value.

Causal interpretation. With $H = 4$ hot partitions and $T = 16$ workers, the optimal mapping assigns exactly one hot partition to each of four distinct threads. `par` achieves this deterministically. Both OpenMP runtimes, in contrast, reach the assignment through a runtime race for the first T work units. If the four hot partitions are claimed by four distinct threads, the run lands in fast mode (∼0.88 s); if two end up on the same thread, that thread doubles its load and the run lands in slow mode (∼1.30 s, exactly ∼2× a single hot partition's duration).

The *difference* between `loop-d1` and `omp_task` lies in how the race is resolved. `loop-d1` dispatches iterations through a global atomic counter: the master thread, entering first, typically wins the initial claims and accumulates small-index hot partitions, biasing the run toward slow mode. `omp_task`, instead, has the master enqueue all P tasks across per-thread deques before the stealers activate, naturally spreading the four hot tasks across distinct threads.

Implication. Bimodality is the genuine behaviour of OpenMP runtimes under strong skew when $H \leq T$, not a measurement artefact. Median values from 3-run benchmarks would have hidden it entirely; only deterministic work assignment, as in `par`, removes the variance.

7 Conclusions

On uniform input the OpenMP and C++ thread implementations are equivalent, peaking at ∼4.95× speedup at $T = 16$ before flattening against the memory wall. Skew exposes the real differences. `omp_loop` with `schedule(static)` stalls at 1.34×, and the cheaper alternatives `dynamic,4` and `guided` fail too: only `dynamic,1` distributes hot partitions across distinct threads. The 30-repetition study further reveals that both OpenMP runtimes are bimodal under strong skew, with median performance dependent on a runtime race that the deterministic stride-T mapping of the C++ thread version sidesteps entirely. The practical guidance is therefore: on balanced workloads the choice of OpenMP construct is immaterial; on skewed workloads the choice of *scheduling policy* dominates the choice of *construct*, and any runtime-mediated distribution carries an inherent variance that explicit static assignment avoids.